

Development / Reflection Phase Submission

StudyFlow: An Academic Workload Planning System for University Students

Course: Project: Software Engineering (DLMCSPSE01)

1. Design and Implementation Procedure

The development of StudyFlow followed an iterative and feature-driven implementation process. After the conception phase, the first step was to establish the technical foundation of the system. FastAPI was selected as the backend framework, Jinja2 was used for server-rendered pages, SQLAlchemy was chosen for database interaction, and SQLite was used for local persistence during development. This setup allowed the application to be executed in a browser from an early stage and made it possible to validate features incrementally.

The implementation then proceeded in a dependency-based order. Authentication and session-based access were introduced first so that all later features could be linked to individual users. After this, subject management and assignment management were implemented, including create, edit, delete, and filtering functionality. Once the core academic data model was stable, the dashboard was added to provide a quick overview of overdue, due-soon, and completed assignments.

In the next iteration, timetable import through .ics files was implemented. The system was extended to parse calendar files, store class events, and derive candidate subjects from uploaded timetable data. After this, recommendation logic and planner functionality were introduced so that due dates, priorities, estimated study hours, and class schedule intensity could be analyzed together. Finally, a weekly calendar view was integrated and generated study sessions were made movable and resizable in order to provide a more practical and interactive planning experience.

This implementation sequence proved effective because each stage produced a usable intermediate version of the system while still leaving room for refinement. The iterative approach also made it easier to incorporate interface improvements and new planning features without needing to redesign the application from scratch.

2. Requirement Refinements During Development

Compared to the conception phase, the requirements became more specific during development. The basic requirement of managing assignments was extended to include editing, filtering, dashboard summaries, and subject-linked academic organization. Calendar integration also evolved beyond simple file import: uploaded timetable data was stored as class events and used as the basis for subject extraction, recommendation logic, planner generation, and calendar visualization.

The planning-related requirements were refined the most. Instead of only displaying tasks and deadlines, the system now generates recommendations, creates suggested study sessions, and displays those sessions alongside lectures in a weekly calendar view. This means that the implemented system goes beyond static task tracking and supports practical workload planning.

3. Technological Approach

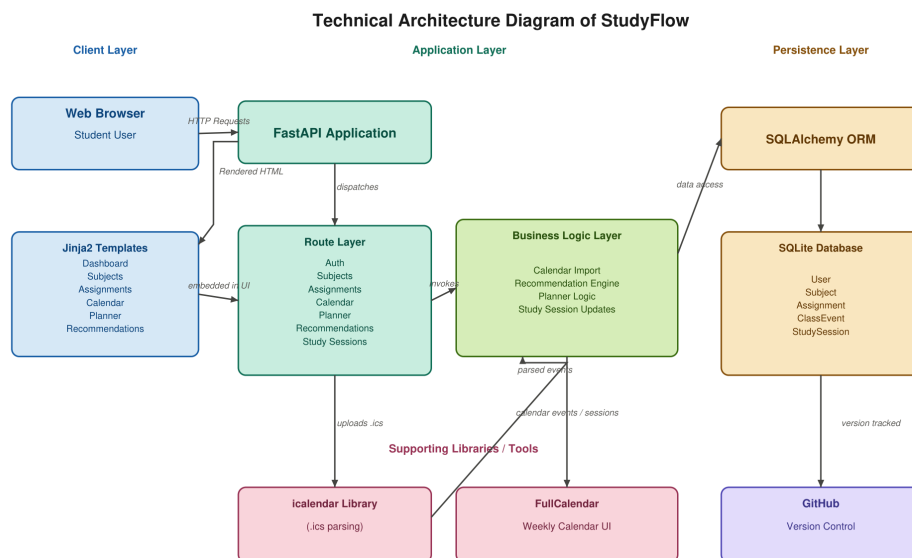
The technological approach was guided by the need to create a browser-based application with a clear structure and manageable complexity. FastAPI was selected because it provides a lightweight and modular backend framework that is well suited to structured route handling and rapid feature development. Jinja2 was chosen for the user interface because server-rendered templates allowed the project to remain focused on core functionality without the additional overhead of a large frontend framework.

SQLAlchemy was used as the object-relational mapper to manage entities such as users, subjects, assignments, class events, and study sessions. SQLite was selected for development because it is easy to configure and appropriate for a course-scale web application. The icalendar library was introduced to parse .ics timetable files, while FullCalendar was integrated later to replace a less intuitive list-based calendar representation with a more realistic weekly calendar interface. GitHub was used for version and release control so that implementation progress and code changes remained traceable throughout the project. These technology choices supported the project goal of balancing functionality, maintainability, and usability.

4. Complemented Architecture

During development, the initial architectural idea was expanded into a clearer layered structure. The implemented system consists of a presentation layer, an application layer, and a persistence layer. The presentation layer contains the browser-based views such as the dashboard, subjects page, assignments page, calendar page, recommendations page, planner, and weekly calendar view. The application layer contains the route handlers and business logic for authentication, calendar import, recommendation generation, planner logic, and study session updates. The persistence layer stores all relevant data in the database through SQLAlchemy models.

The complemented architecture also reflects a clearer separation of concerns. Calendar import handles timetable processing, recommendation logic evaluates workload and urgency, planner logic transforms this into study sessions, and the weekly calendar view provides a visual and interactive representation of the result. The technical architecture diagram below shows this separation between client layer, application layer, persistence layer, and supporting libraries such as icalendar and FullCalendar.



Technical Architecture Diagram of StudyFlow. The browser-based client layer renders Jinja2 templates, interacts with the FastAPI application through HTTP requests, and embeds the FullCalendar interface for weekly scheduling. The application layer contains route handlers and business logic for calendar import, recommendation generation, planner logic, and study session updates. Persistent information is managed through SQLAlchemy and stored in the SQLite database. Supporting libraries and tools include the icalendar package for .ics parsing and GitHub for version control.

5. Key Design Decisions and Reflection

Several design decisions were refined during implementation. Calendar-based subject import was implemented as a user-controlled step rather than a fully automatic process, because timetable titles

may vary in format and quality. Recommendation and planner functionality were kept rule-based instead of being implemented as a more complex optimization engine, since a transparent and understandable planning model was more appropriate for the project scope. In the weekly calendar, class events were kept fixed while study sessions were made adjustable, because this reflects the practical difference between scheduled lectures and flexible study time.

A further important refinement was the introduction of FullCalendar. Earlier list-based schedule displays were functional, but not intuitive enough to communicate time blocking clearly. Replacing them with a true weekly calendar significantly improved usability and made the planning functionality more tangible.